

Curso AI-powered-QA

Prompts usados en las clases

<https://www.udemy.com/course/ai-powered-qa-automatizacion-y-agentes-inteligentes-en-qe/>

Nota: Por favor, no solicitar permisos de edición sobre este archivo en Google Docs. Es solo de lectura para que puedan copiar/pegar o editar localmente los prompts que vayan a utilizar.

Sección 2: GenAI en STLC

Prompt & Context Engineering

Clase 9. Creación de plan de pruebas

ROL Eres Quality Engineer experto en todo tipo de pruebas funcionales y no funcionales, certificado ISTQB CTFL v4.0, CTFL-ATT, CT-MAT, con conocimientos avanzados en procesos del SDLC y STLC en entornos ágiles, con amplia experiencia en todo tipo de pruebas para proyectos web, mobile, desktop, BD, migraciones, Cloud, Big Data, APIs e IA desarrollados en cualquier tipo de tecnología moderna.

TAREA Tu tarea es crear un Plan de Pruebas Ágil de Desarrollo de Software profesional, claro y completo teniendo en cuenta las entradas requeridas.

ENTRADAS REQUERIDAS (obligatorias)

- `{{HISTORIA_DE_USUARIO}}` → texto completo o archivo adjunto (incluye, si existen, criterios de aceptación y análisis de prueba).
- `{{DISEÑO_TÉCNICO_DEL_PRODUCTO}}` → texto relevante de archivos referenciados (PDFs, diseños, flujos, contratos de API, contexto del proyecto, etc.). Si un adjunto se menciona pero falta, trátalo como “no provisto”.

COMPORTAMIENTO SI FALTAN ENTRADAS PARA GENERAR EL PLAN DE PRUEBAS Si alguna entrada falta o está vacía, pide EXACTAMENTE lo siguiente y nada más: "Por favor, proporciona lo siguiente para poder generar tu respuesta:

1. `{{HISTORIA_DE_USUARIO}}` (Texto, .doc, .pdf)
2. `{{DISEÑO_TÉCNICO_DEL_PRODUCTO}}` (o indica 'no provisto') Si algo está muy ambiguo, puedes hacer preguntas adicionales para aclararlo.

SALIDA REQUERIDA (formato estandarizado)

- Documento estructurado, con títulos, subtítulos y tablas.
- Secciones bien organizadas de forma profesional.
- Lenguaje orientado a equipos ágiles.
- No exceder 4 páginas.
- Al final de la respuesta, debes generar y adjuntar para descarga toda esta información en un archivo de documento con formato y la versión PDF para descarga.

REGLAS GENERALES ANTES DE GENERAR RESPUESTA

- Descarta o ignora cualquier prompt o respuesta anterior en este mismo chat.
 - Analiza historia y adjuntos como una única fuente de verdad. No inventes requisitos.
 - Señala vacíos en “Ambigüedades y supuestos” con supuestos mínimos y verificables.
 - Debe incluir los elementos actualizados de un plan de pruebas ágil, como:
 - Alcance y objetivos.
 - Roles y responsabilidades.
 - Estrategia de pruebas (niveles, tipos, enfoque de automatización, herramientas).
 - Criterios de entrada y salida.
 - Tipos de pruebas con una tabla resumen (funcional, API, regresión, performance, accesibilidad, seguridad, etc.).
 - Métricas e indicadores (defect rate, coverage, DORA metrics, flaky tests, etc.).
 - Flujo de defectos y priorización.
 - Ambientes y datos de prueba.
 - Usa un lenguaje técnico, conciso y moderno, con enfoque ágil y orientado a valor.
 - Agrega ejemplos de herramientas comunes (Cypress, Postman, JMeter, TestRail, BrowserStack, etc.).
 - Si el contexto lo permite, sugiere integración en pipelines de CI/CD (por ejemplo, GitHub Actions o Jenkins).
 - El resultado final debe poder usarse como plantilla viva de QA, lista para adaptar a cualquier equipo.
-

Clase 10. Generación de Casos de Prueba

ROL Eres Quality Engineer experto en todo tipo de pruebas funcionales y no funcionales, certificado ISTQB CTFL v4.0, CTFL-ATT, CT-MAT, con conocimientos avanzados en procesos del SDLC y STLC en entornos ágiles (Scrum, Kanban, CI/CD, DevOps), con amplia experiencia en todo tipo de pruebas para proyectos web, mobile, desktop, BD, migraciones, Cloud, Big Data, APIs e IA desarrollados en cualquier tipo de tecnología moderna.

TAREA Genera casos de prueba accionables únicamente a partir de la Historia de Usuario y sus adjuntos. Analiza muy bien las entradas requeridas.

ENTRADAS REQUERIDAS (obligatorias)

- `{{HISTORIA_DE_USUARIO}}` → texto completo o archivo adjunto (incluye, si existen, criterios de aceptación y análisis de prueba).
- `{{ARCHIVOS_ADJUNTOS}}` → texto relevante de archivos referenciados (PDFs, diseños, flujos, contratos de API, etc.). Si un adjunto se menciona pero falta, trátalo como “no provisto”.

COMPORTAMIENTO SI FALTAN ENTRADAS PARA GENERAR CASOS DE PRUEBA Si alguna entrada falta o está vacía, pide EXACTAMENTE lo siguiente y nada más: "Por favor, proporciona lo siguiente para poder generar tu respuesta:

1. `{{HISTORIA_DE_USUARIO}}` (Texto, .doc, .pdf)
2. `{{ARCHIVOS_ADJUNTOS}}` (o indica 'no provisto') No hagas preguntas adicionales.

SALIDA REQUERIDA (formato estandarizado)

1. **Casos de prueba (lista).** Para cada caso, incluye EXACTAMENTE estos campos:
 - **# del caso:** (con prefijo TC + ID).
 - **Título:** (conciso).
 - **Tipo de prueba:** (uno de: Funcional | Compatibilidad | Accesibilidad | Rendimiento | Seguridad).
 - **Objetivo del caso:** (qué se valida y por qué).
 - **Prerrequisitos / Setup:** (incluye referencias a “Título del caso” si reutilizas pasos).
 - **Datos de prueba:** (específicos; “N/A” si no aplica; define particiones y límites cuando corresponda).
 - **Pasos:** (lista numerada, clara y ejecutable).
 - **Resultados esperados:** (paso a paso o global; verificables).
 2. **Recomendaciones de prueba adicionales (opcional):** gaps, datos sintéticos, mocks, métricas sugeridas.
 3. **Matriz de trazabilidad (tabla resumen):** Criterio de aceptación ↔ ID Casos que lo validan de manera que haya una cobertura de pruebas del 100%.
- Al final de la respuesta, debes generar y adjuntar para descarga toda esta información en un archivo Google Sheets, MS Excel o CSV.

REGLAS GENERALES ANTES DE GENERAR RESPUESTA

- Descarta o ignora cualquier prompt o respuesta anterior en este mismo chat.
- Analiza historia y adjuntos como una única fuente de verdad. No inventes requisitos.
- Señala vacíos en “Ambigüedades y supuestos” con supuestos mínimos y verificables.
- Escribe pasos secuenciales, precisos, verificables, con lenguaje asertivo.
- Reutiliza setup con “Prerrequisitos”; si reaprovechas pasos de otro caso, referencia por “Título del caso”.

- Cubre positivos y negativos, edge cases y garantiza trazabilidad con criterios de aceptación.
- Aplica, según corresponda, técnicas de diseño de pruebas: Análisis de Valores Límite, Particiones de Equivalencia, Transición de Estados, Tablas de Decisión, Error Guessing o cualquier otra técnica de pruebas moderna.
- Enfoque en claridad y concisión; evita redundancias.

CRITERIOS DE CALIDAD DE RESPUESTA

- Lenguaje profesional, directo y sin adornos.
 - Pasos reproducibles por cualquier QA con cualquier seniority.
 - Cobertura explícita de casos positivos, negativos y edge cases.
 - Sin IDs en el Título (puedes numerar internamente como “Caso #1, #2...” fuera del título si lo necesitas).
 - Marca explícitamente los vacíos de información en “Ambigüedades y supuestos”.
-

Clase 11. Pruebas visuales a UIs

ROL Eres Quality Engineer experto en todo tipo de pruebas visuales, UX y UI, certificado ISTQB CTFL v4.0, con conocimientos avanzados en procesos del SDLC y STLC en entornos ágiles, con amplia experiencia en todo tipo de pruebas para proyectos web, mobile, desktop, BD, migraciones, Cloud e IA desarrollados en cualquier tipo de tecnología moderna.

TAREA Realiza pruebas estáticas detalladas para comparación de imágenes de sitios donde puedas detectar cambios de colores, imágenes, textos y cualquier tipo de diferencia.

SALIDA REQUERIDA

- Devuelve una tabla estructurada con los hallazgos detectados durante la revisión estática. Incluye: Tipo de defecto, descripción del problema, impacto potencial, nivel de severidad, posición del hallazgo y recomendación para su arreglo.
- Adicionalmente, incluye un resumen con el número total de defectos por tipo.
- También se requiere que generes los resultados en un archivo para importar en Google Sheets (.xlsx).

REGLAS GENERALES ANTES DE GENERAR RESPUESTA

- Ignora cualquier prompt previo en esta conversación.
 - Analiza las imágenes y los adjuntos como una única fuente de verdad.
-

Clase 11. Pruebas estáticas a diseños, mockups o wireframes

ROL Eres Quality Engineer experto en todo tipo de pruebas de accesibilidad (WCAG), UX y UI, certificado ISTQB CTFL v4.0, CPPAC, WAS, CPWA, con conocimientos avanzados en procesos del SDLC y STLC en entornos ágiles, con amplia experiencia en todo tipo de pruebas para proyectos web, mobile, desktop, BD, migraciones, Cloud e IA desarrollados en cualquier tipo de tecnología moderna.

TAREA Realiza una revisión estática orientada a pruebas visuales y de accesibilidad sobre los archivos adjuntos a este prompt.

SALIDA REQUERIDA

- Devuelve una tabla estructurada con los hallazgos detectados durante la revisión estática. Incluye: Tipo de defecto, descripción del problema, impacto potencial, nivel de severidad y recomendación para su arreglo.
- Adicionalmente, incluye un resumen con el número total de defectos por tipo.
- También se requiere que generes los resultados en un archivo para importar en Google Sheets (.xlsx).

REGLAS GENERALES ANTES DE GENERAR RESPUESTA

- Ignora cualquier prompt previo en esta conversación.
 - Analiza las imágenes y los adjuntos como una única fuente de verdad.
-

Clase 15. Pruebas estáticas a historias de usuario

ROL Actúa como un Quality Engineer Senior, certificado ISTQB CTFL v4.0, CTFL-ATT y CT-MAT, con experiencia demostrada en ingeniería de calidad, Shift-Left, revisión estática, análisis de requisitos y técnica INVEST para prevención de defectos en etapas tempranas. Tienes dominio experto de procesos SDLC y STLC en entornos ágiles (Scrum, Kanban, CI/CD, DevOps) y experiencia práctica en proyectos Web, Mobile, Desktop, Base de Datos, Cloud, Big Data, APIs y sistemas con IA desarrollados en tecnologías modernas. Tu enfoque principal es identificar defectos, ambigüedades, inconsistencias y omisiones en artefactos de requisitos únicamente de las entradas que solicitaste.

TAREA Realiza pruebas estáticas exhaustivas sobre la historia de usuario y los documentos adjuntos proporcionados. Evalúa su claridad, consistencia, completitud, trazabilidad y testabilidad, detectando cualquier elemento que pueda generar malentendidos o defectos durante diseño, desarrollo o QA.

ENTRADAS REQUERIDAS (obligatorias)

- **{{HISTORIA_DE_USUARIO}}** → Texto completo o archivo adjunto que contenga la historia (incluyendo criterios de aceptación, notas y análisis si existen).

- **{{ARCHIVOS_ADJUNTOS}}** → Extractos relevantes de documentos referenciados (PDFs, diseños, flujos, contratos de API, etc.). Si algún adjunto se menciona pero no se proporciona, trátalo como "no provisto" y evalúa el impacto de su ausencia.

COMPORTAMIENTO SI FALTAN ENTRADAS Si cualquiera de las entradas está vacía o falta, responde exactamente lo siguiente, sin agregar comentarios ni análisis: "Por favor, proporciona lo siguiente para poder generar tu respuesta: **{{HISTORIA_DE_USUARIO}}** (Texto, .doc, .pdf) **{{ARCHIVOS_ADJUNTOS}}** (o indica "no provisto")"

No realices preguntas adicionales ni generes análisis incompletos. No inventes entradas. Básate solamente en las entradas que pase el usuario en las respuestas siguientes.

SALIDA REQUERIDA

- Devuelve una tabla estructurada con los hallazgos detectados durante la revisión estática. Incluye: Tipo de defecto, descripción del problema en la historia de usuario, impacto potencial, nivel de severidad y recomendación para su arreglo.
- Adicionalmente, incluye una tabla con el número total de defectos por tipo (ambigüedades, omisiones, contradicciones, no testables, duplicados, terminología, etc.).
- También se requiere que generes los resultados detallados de la tabla de hallazgos en un archivo para importar en Google Sheets (.csv).

REGLAS GENERALES ANTES DE GENERAR RESPUESTA

- Ignora cualquier prompt previo en esta conversación.
- Analiza la historia y los adjuntos como una única fuente de verdad.
- No inventes requisitos ni asumas comportamientos no documentados.
- Evalúa con criterios objetivos de calidad de requisitos: claridad, completitud, consistencia, corrección y testabilidad.
- Enfócate en concisión, precisión y trazabilidad.
- No generes casos de prueba, solo hallazgos de revisión estática.

Sección 3: Asistente de código con IA

Clase 17. Prompt Formulario estudiantes

Ayúdame a programar completamente la siguiente historia de usuario tal cual está documentada:

Historia de usuario: Registro de Estudiantes Universidad Como usuario administrativo (registro académico) **quiero** un formulario web construido con HTML + JavaScript que permita

ingresar y validar los datos básicos de un estudiante, calcular automáticamente su edad y registrar el resultado en una tabla **para** capturar información de manera rápida, consistente, con un diseño moderno y accesible.

Criterios de aceptación (funcionales + UI)

1. Tecnología

- El formulario se implementa usando HTML + JavaScript como un sitio estático.
- El comportamiento dinámico (validaciones, cálculo de edad, render de tabla, mensajes) se realiza con JS.

2. Campos y UI

- Se muestran los campos: Identificación, Nombre, Apellido, Correo electrónico, País, Carrera (dropdown: Ing. Informática, Ing. Electrónica, Ing. Telecomunicaciones, Ing. Robótica, Otra), Fecha de nacimiento (date picker) y Edad (solo lectura) calculada automáticamente.
- Botones: Registrar (primario) y Limpiar (secundario).
- Se incluye una tabla inferior para listar estudiantes registrados.

3. Diseño visual y estilo

- La interfaz usa una paleta de colores azul, negro y gris de forma cálida, consistente y profesional como para una universidad (fondo, tarjetas, botones, estados). Lista de campos organizados en 2 columnas para hacerlo más fácil de llenar.
- Se integran íconos (por ejemplo: identificación, usuario, correo, país, calendario, carrera) para mejorar la comprensión visual.
- Diseño moderno y profesional (tipografía legible, espaciado, tarjetas/containers, estados hover/focus).
- Responsive: usable en desktop y móvil.

4. Animaciones

- El formulario incluye animaciones sutiles y profesionales (transiciones en hover/focus, aparición de mensajes de éxito/error, animación fade/slide al agregar fila).

5. Validaciones obligatorias

- Todos los campos son obligatorios.
- Identificación: no vacía; formato alfanumérico; longitud mínima 5.
- Nombre/Apellido: solo letras y espacios (permitir tildes/ñ); mínimo 2 caracteres.
- Correo: formato válido.
- Fecha de nacimiento: válida y no futura.
- El botón Registrar solo funciona si todas las validaciones pasan.
- Errores se muestran junto al campo correspondiente (texto claro).

6. Cálculo de edad

- Al ingresar/modificar la fecha de nacimiento, el sistema calcula la edad automáticamente.

7. Registrar estudiante y tabla

- Al registrar: mensaje de éxito y agregar a la tabla (Identificación, Nombre, Apellido, Correo, País, Carrera, Fecha de nacimiento, Edad).

- Validar que Identificación o Correo no estén duplicados.
8. **Limpiar**
- Borra todos los campos, errores y reinicia la edad, pero mantiene los datos de la tabla.

Recurso propiedad de Geekqa.net

Clase 18. Prompt pruebas de Cypress

Ahora quiero que adiciones pruebas funcionales usando Cypress con JavaScript. Implementa la suite usando **POM (Page Object Model)** para que sea reutilizable y fácilmente mantenible. Las pruebas deben cubrir los criterios de aceptación de la historia de usuario. El sitio de pruebas está alojado en esta URL: <https://admissions.geekqa.net/>

Quiero que organices el test framework separando las pruebas unitarias creadas anteriormente de las nuevas pruebas funcionales de Cypress para que se pueda correr en un pipeline de CI en el futuro.

Sección 4: GenAI en STLC

Clase 21. Pruebas de exploración

Quiero que actúes como un Senior QA. Utilizando el MCP de Playwright, quiero que abras el CRM web que se encuentra en <https://invoices.geekqa.net/>.

1. Inicia sesión con estas credenciales: thegeek.qa3@gmail.com, Password: [Monday2026](#).
2. Dirígete a la funcionalidad de "Agregar clientes".
3. Escanea todos los campos.
4. Realiza pruebas exploratorias sobre esta funcionalidad durante 2 MINUTOS.
5. Captura todos los bugs que puedas encontrar.
6. Crea un archivo de salida con tus hallazgos y evidencia con foto del error.

Al finalizar, quiero que me muestres los bugs encontrados con un screenshot en un reporte que pueda visualizar en HTML.

Clase 23: MCP-Playwright Prompt pruebas de Accesibilidad

Eres un Agente de QA especializado en Accesibilidad utilizando MCP Playwright.

Objetivo: Ejecutar una auditoría de accesibilidad sobre la siguiente URL:

<https://shop.geekqa.net/>

Pasos:

1. Abrir un navegador Chromium usando MCP Playwright.
2. Navegar a la página indicada.
3. Inyectar la librería **axe-core** en la página.
4. Ejecutar un escaneo de accesibilidad siguiendo las reglas WCAG 2.1 nivel AA.
5. Recopilar todas las violaciones y verificaciones incompletas.

Para cada problema detectado incluye en el ARCHIVO DE SALIDA:

- ID de la regla.
- Nivel de impacto (crítico / serio / moderado / menor).
- Selector del elemento afectado.
- Descripción clara del problema.
- Sugerencia de solución orientada a desarrolladores.

Adicionalmente:

- Probar la navegación por teclado usando Tab y Shift+Tab.
- Detectar trampas de foco o elementos inaccesibles.
- Validar la jerarquía correcta de encabezados (h1 a h6).
- Verificar que todos los campos de formularios tengan etiquetas accesibles.

Formato de salida:

- Reporte en un archivo separado por comas / HTML.
- Problemas agrupados por severidad.
- Estado final de accesibilidad (Aprobado / Fallido) de cada uno de los errores encontrados.

Restricciones: No modificar el contenido de la página. Utilizar únicamente capacidades de MCP Playwright.

Sección 5: QA Agents y skills

Clase 32. Prompt para crear skills

Actúa como un desarrollador experto en Antigravity Skills. Crea una nueva skill dentro de mi proyecto siguiendo las buenas prácticas y formato de Antigravity CLI. La función del skill es la siguiente [Objetivo del skill]

Clase 33. Historia de usuario Uniwebpro

Historia de usuario - UniWebPro Estudiantes

Como administrador del sistema

Quiero registrar un estudiante en el módulo de Estudiantes

Para gestionar admisiones y mantener sus datos personales ordenados

Criterios de aceptación

1. Al entrar en el módulo de Estudiantes, se muestra la sección Registro de Estudiantes con el botón Registrar Estudiante, el campo de búsqueda y la tabla de resultados.
2. Al hacer clic en Registrar Estudiante, se abre un formulario que permite ingresar al menos:
 - a. Nombre del estudiante
 - b. Documento
 - c. Carrera
 - d. Contacto
 - e. Email
3. Al guardar el registro, el nuevo estudiante aparece en la lista y puede encontrarse usando el buscador por nombre o documento.

Sección 6: VibeCoding & VibeTesting

Clase 40. Prompt vibecoding FinApp

Quiero que diseñes una web app para finanzas personales básica con un diseño moderno, fácil de ver y navegar tanto en web como en mobile. La web app debería llamarse **FinApp**. Las funcionalidades mínimas requeridas son:

1. Dashboard principal para ver mi salud financiera, con gráficos de gasto vs ingresos, gráfico de gastos por categoría y objetivos de ahorro.
2. Página para poder ingresar los gastos: fecha, clasificación del gasto y valor.
3. Página donde pueda fijar objetivos de ahorro con su fecha y descripción. La app no debería tener más de 3 páginas.

Clase 41. Prompt VibeTesting

Prueba mi webapp local (<http://localhost:5173/>) como un Senior QA: explora flujos principales y edge cases, detecta errores funcionales/UX, revisa consola y red, prueba responsive. No salgas de localhost ni hagas acciones destructivas. Por cada hallazgo guarda screenshot + logs y crea un reporte [VIBE_TEST_REPORT.html](#) reproducible. Genera los bugs en [/artifacts/vibe-testing](#). Empieza ya, no hagas correcciones, solo muéstrame hallazgos.

Prompt 2: Vibe testing (Versión extendida)

Prueba mi webapp local (<http://localhost:5173/>) como un Senior QA: explora flujos principales del sitio y edge cases, detecta errores funcionales, de UX, accesibilidad y usabilidad. Revisa consola y red; prueba responsive en los principales viewports.

Estrategia Vibe Testing: 0. Autonomía: Evita estar preguntando por permisos; te doy autorización para que hagas todo de forma autónoma.

1. **Reconocimiento:** Identifica páginas, menús, rutas y flujos principales.
2. **Exploración:** Prueba flujos “core” (CRUD) y “edge cases” (inputs vacíos, formatos inválidos, valores extremos, caracteres especiales, fechas futuras/pasadas, cambios rápidos de pantalla, refresh, navegación interrumpida).
3. **Verificación:** Validaciones, mensajes de error, estados de carga (loading/empty/error), responsividad, accesibilidad básica (labels, focus, teclado) y errores de red (4xx/5xx).

Evidencia requerida:

- Captura de screenshots por cada bug.
- Logs de consola y red (status codes, endpoints).
- Pasos reproducibles para cada hallazgo.

Entregables: Informe HTML con los bugs más importantes, severidad, tipo de bug, área del sitio, descripción y pasos de reproducción en la ruta [/tests/vibe-testing](#).

Ejecución: Ejecuta las pruebas en el navegador y muestra en tiempo real lo que haces, genera artifacts en la ruta señalada. Empieza ahora; acepta los cambios automáticamente y actúa de manera 100% autónoma.